

RICE UNIVERSITY

EDES301: INTRODUCTION TO PRACTICAL ELECTRICAL
ENGINEERING

Assignment #3

Command Line, Disk Space, and Binary Systems

Student Name:

Sandeep Ramlochan

Lecturer:

Erik Welsh

Due Date:

September 30, 2025 (8pm)

1 Command Line (Cloud9 IDE Screenshots)

- 1a. Clone repository and make directory:

Commands: `git clone ...` and `mkdir assignment_03`.

```
Cloning into '/var/lib/cloud9/EDES301'...
remote: Enumerating objects: 24, done.
remote: Counting objects: 100% (24/24), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 24 (delta 4), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (24/24), done.
```

Figure 1: Cloning the EDES301 repository and creating the `assignment_03` directory.

- 1b. Change directory:

Command: `cd EDES-301-REAL`.

```
debian@beaglebone:/var/lib/cloud9$ cd EDES-301-REAL
debian@beaglebone:/var/lib/cloud9/EDES-301-REAL$ mkdir assignment_03
mkdir: cannot create directory 'assignment_03': File exists
```

- 1c. Create file and 1d. Add "Hello World!" to the file:

Command: `touch temp.txt` (to create the file) followed by `echo "Hello World!" > temp.txt` (to add content).

```
debian@beaglebone:/var/lib/cloud9/EDES-301-REAL/assignment_03$ touch temp.txt
```

```
GNU nano 3.2
Hello World!
```

- 1e. Concatenate the file to the command line:

Command: `cat temp.txt`.

```
Hello World!
```

Figure 2: Hello World on `temp.txt`

- **1f. Perform a long listing of all the files:**

Command: `ls -l`.

```
debian@beaglebone:/var/lib/cloud9/EDES-30
temp.txt
debian@beaglebone:/var/lib/cloud9/EDES-30

debian@beaglebone:/var/lib/cloud9/EDES-
total 12
drwxr-xr-x 2 debian debian 4096 Dec  1
drwxr-xr-x 6 debian debian 4096 Dec  1
-rw-r--r-- 1 debian debian  13 Dec  1
```

2 Increase Disk Space (Cloud9 IDE Screenshots)

- 2a. Check your current disk space:

Command: `df -h`.

```
debian@beaglebone:/var/lib/cloud9/EDES-301-REAL/
Filesystem      1K-blocks      Used Available Use%
udev            218884         0    218884   0%
tmpfs           49520         264     49256   1%
/dev/mmcblk0p1  3544568  2268220    1075852  68%
tmpfs           247588         0    247588   0%
tmpfs           5120          0       5120   0%
tmpfs           247588         0    247588   0%
```

- 2b. Run "grow_partition.sh" and take a picture of your current disk space:

Commands: `sudo /opt/scripts/tools/grow_partition.sh` followed by `df -h`.

```
debian@beaglebone:/var/lib/cloud9/EDES-301-REAL/assignment_03$ sudo /opt/scripts/tools/grow_partition.sh
[sudo] password for debian:
Media: [/dev/mmcblk0]
sfdisk: 2.26.x or greater
Disk /dev/mmcblk0: 29.1 GiB, 31267487744 bytes, 61069312 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x5ea0f889

Old situation:

Device      Boot Start      End Sectors  Size Id Type
/dev/mmcblk0p1 *      8192 7372799 7364608   3.5G 83 Linux

>>> Created a new DOS disklabel with disk identifier 0xa0a508e4.
/dev/mmcblk0p1: Created a new partition 1 of type 'Linux' and of size 29.1 GiB.
Partition #1 contains a ext4 signature.
/dev/mmcblk0p2: Done.

Partition #1 contains a ext4 signature.
/dev/mmcblk0p2: Done.

New situation:
Disklabel type: dos
Disk identifier: 0xa0a508e4

Device      Boot Start      End Sectors  Size Id Type
/dev/mmcblk0p1 *      8192 61069311 61061120 29.1G 83 Linux

The partition table has been altered.
Calling ioctl() to re-read partition table.
WARNING: old partition table is busy.
The kernel still uses the old table. The new table will be used at the next reboot or after you run partprobe(8) or kpartx.
Syncing disks.
```

Figure 3: Enter Caption

3 Installing Packages (PocketBeagle)

Note: This section outlines the required procedures.

- 3a. Update Debian Sources:

`archive.debian.org`).

- 3b. Update package repository:

Command: `sudo apt-get update`.

- 3c. Install a library (zip):

Command: `sudo apt-get install -y zip`.

- 3d. Install EDES 301 Libraries / Software:

Procedure: Download `edes301-linux-setup.sh`, move to `/var/lib/cloud9/`, change permissions (`chmod 755 edes301-linux-setup.sh`), and execute (`sudo ./edes301-linux-setup.s`

4 Base Conversion

Express the following decimal numbers in Binary, Octal, and Hexadecimal.

- **4a. 64,206**

- Binary: 1111101000001110_2
- Octal: 175016_8
- Hexadecimal: $FA0E_{16}$

- **4b. 48,879**

- Binary: 1011111011101111_2
- Octal: 137357_8
- Hexadecimal: $BEEF_{16}$

- **4c. 57,005**

- Binary: 1101111011101101_2
- Octal: 157355_8
- Hexadecimal: $DEED_{16}$

5 Binary Representations

Express the following numbers as Decimal (Base 10) or ASCII Characters.

5a. Unsigned Binary Integer (Decimal)

- **5a.i.** $0x2A$ (8-bit)
 $00101010_2 = 42_{10}$
- **5a.ii.** $0xFF$ (8-bit)
 $11111111_2 = 255_{10}$
- **5a.iii.** $0111\ 1110\ 1000$
 2024_{10}
- **5a.iv.** $1010\ 1100\ 1110\ 1101$
 44269_{10}

5b. 2's Complement Signed Integer (Decimal)

- **5b.i.** 0x2A (8-bit)
 $00101010_2 = 42_{10}$
- **5b.ii.** 0xFF (8-bit)
 $11111111_2 = -1_{10}$
- **5b.iii.** 0111 1110 1001
 2025_{10}
- **5b.iv.** 1010 1100 1110 1101
 -21267_{10}

5c. ASCII Characters

- **5c.i.** 0x49
 $0x49 = 73_{10} \rightarrow \text{'I'}$
- **5c.ii.** 0110 1100 0110 1111 0111 0110 0110 0101
 $\text{'l', 'o', 'v', 'e'} \rightarrow \text{'love'}$
- **5c.iii.** 0x45444553
 $\text{'E', 'D', 'E', 'S'} \rightarrow \text{'EDES'}$
- **5c.iv.** 3,354,673
 $3,354,673_{10} = 0x334531$. Interpreting the hexadecimal bytes as ASCII:
 $0x33 \rightarrow \text{'3'}$
 $0x45 \rightarrow \text{'E'}$
 $0x31 \rightarrow \text{'1'}$
 $\rightarrow \text{'3E1'}$

The aspect of PEP 8 I find most restrictive and occasionally disagree with is the strict recommendation of a **79-character line limit**.

While the spirit of this rule—to enhance readability, especially on older terminals or when viewing multiple files side-by-side—is admirable, its rigid enforcement can sometimes hinder modern development practices. On contemporary wide-screen monitors, an 80-character limit often forces the developer to break up long, logically cohesive expressions or function calls onto multiple lines using implicit line continuation (parentheses). When this continuation is applied to arguments that are already deeply nested, it creates vertical sprawl, sometimes reducing, rather than increasing, the immediate clarity of the code block.

For example, a clear, single-line mathematical expression must be broken, or a descriptive variable name must be shortened, solely to satisfy the width constraint. In collaborative environments utilizing modern IDEs with automatic wrapping and code folding, I believe a slightly more lenient boundary, such as 90 or 100 characters (as adopted by some large projects), offers a better balance between maintaining vertical density and ensuring readability without sacrificing the natural structure of the logic. I agree with the intent, but the specific number feels outdated for 2025.

Do you agree that whitespace should matter in Python and why?

Yes, I **strongly agree** that whitespace (specifically indentation) should matter in Python, as it is one of the language’s most significant design strengths and a fundamental reason why Python code is generally considered highly readable.

Python uses mandatory indentation to define block structure (such as function bodies, loops, and conditional statements), replacing the curly braces (`{}`) found in languages like C++ or Java. This feature, known as significant whitespace, forces the visual layout of the code to directly correspond to its logical structure. This eliminates the common issue in brace-delimited languages where poor or inconsistent indentation can mislead the reader about the actual flow of execution.

By making the visual hierarchy of the code mandatory, Python effectively enforces a high standard of consistency across different developers and projects. This drastically reduces the time spent during code reviews on arguing about stylistic conventions, allowing reviewers to focus purely on the logic and correctness of the application. It aligns perfectly with “The Zen of Python,” which states, “Readability counts.” The initial learning curve is negligible compared to the long-term benefits of maintainable and clean codebases.

What is the most interesting thing you found in the style guide and why?

The most interesting and immediately useful rule I encountered was the encouragement of **implicit line continuation using parentheses, brackets, or braces** over explicit line continuation using the backslash (`\`).

The guide clearly states that lines can be wrapped within delimiters without needing a backslash. For example, a long list of arguments or elements can be broken onto multiple lines simply by enclosing them in parentheses, brackets, or braces.

I previously assumed the backslash was the only mechanism for breaking long lines of code. However, the backslash is prone to errors (a space after the backslash causes a syntax error) and visually clutters the code. By contrast, using implicit continuation is far more "Pythonic" and elegant. This technique makes defining complex data structures (like long dictionaries or lists) and calling functions with many descriptive arguments much cleaner, allowing me to naturally align arguments and break lines at logical points while maintaining flow. This rule directly solves the line limit problem in a graceful way, promoting better structure and highly readable code layouts.